ICS 433 | Operating Systems Lab

Phase 2 — Implementation Progress Report

Virtual Memory and Page-Replacement Simulator with Thrashing Analysis
10 November 2026

Team Members

Member	Full name	Student ID
Member 1	Layan Alasmari	202223720
Member 2	Raghad alshammari	202256720
Member 3	Raghad Alsenani	202270580
Member 4	Ghalaa Alshahrani	202266180

1. Project Overview

This project develops a C-based virtual-memory simulator to demonstrate how physical frames are managed and how page-replacement policies affect page faults. The implementation includes First-In, First-Out (FIFO), Least Recently Used (LRU), and Clock (Second Chance) algorithms, with supporting modules for trace input, memory operations, performance metrics, and working-set/thrashing analysis.

2. Implemented Components

Module	Files	Current implementation
FIFO and LRU	src/fifo.c, src/lru.c	Page-hit, page-fault, and replacement counts; five automated scenarios passed.
Clock and memory	src/clock.c, src/memory.c	Second-chance replacement; frame lookup/loading and access/dirty-state operations.
Metrics and thrashing	src/metrics.c, src/thrashing.c	Rate calculations, formatted metrics, working-set estimation, and thrashing-analysis functions.
CLI and trace handling	src/main.c, src/cli.c, src/trace.c	Algorithm/frame selection, manual or file-based trace entry, algorithm dispatch, and metric output.

3. Architecture and Integration Status

The project separates declarations in include/ from C implementations in src/. main.c obtains a reference trace through the CLI/trace modules, converts references into page numbers, invokes the selected replacement algorithm, and passes returned Statistics to the metrics module. A Makefile builds the nine C source files into a simulator executable. The supplied ZIP compiled successfully with GCC using make.

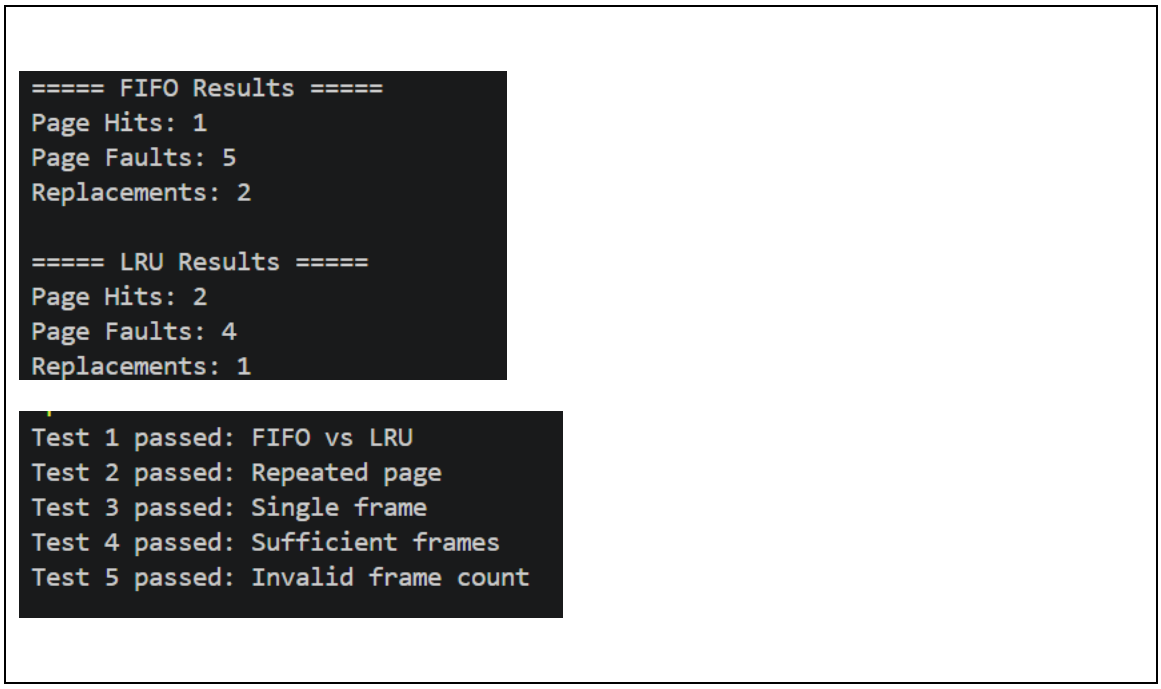
Integration remains in progress: although the main program calls all three replacement algorithms, it currently passes only page numbers into the algorithm wrappers. The shared memory/dirty-page functions and thrashing analysis exist as modules but are not yet fully incorporated into the main

execution/reporting path. In particular, main.c passes zero write-backs to metrics_from_statistics; end-to-end dirty-page and thrashing outputs should not be presented as completed.

4. Testing and Evidence

FIFO/LRU unit tests were executed and all five passed: comparison sequence, repeated page, single frame, sufficient frames, and invalid frame count. For the sequence [1, 2, 3, 1, 4, 1] with three frames, FIFO produced 1 hit, 5 faults, and 2 replacements; LRU produced 2 hits, 4 faults, and 1 replacement. The latest supplied project ZIP also built successfully using its Makefile. Full integrated functional testing is still pending.

FIFO/LRU tests:



sample-run output:



5. Overall Completion and Remaining Work

Estimated Phase 2 implementation progress: approximately 65% complete (team-level estimate, not an automated measurement). Core modules have been written, but the integrated simulator has not yet been fully verified. This estimate should be updated after the final integration run.

Workstream	Completed so far	Still to finish / verify
FIFO / LRU	Both algorithms implemented; five automated tests passed	Connect to common frame state, dirty-page accounting and full trace flow
Clock / memory	Individual Clock and memory-management functions implemented	Verify shared frame state and write-back behavior through integration
Metrics / thrashing	Metrics and thrashing-related functions implemented	Connect to live simulation results; validate working-set and thrashing outputs
CLI / trace / main	CLI, trace parsing and main-program components implemented	Finish end-to-end wiring and test manual/file inputs for all algorithms
Build / quality	Individual modules available; project-level build work underway	Run clean build, complete-system tests, fix defects and capture screenshots

Remaining milestones: (1) complete module integration; (2) validate FIFO, LRU and Clock on the same traces; (3) verify dirty-page write-backs, metrics and thrashing detection; (4) test invalid inputs and multi-process behavior against the proposal; (5) capture final screenshots and finalize the submission.

6. Challenges and Next Steps

The main challenge is combining independently developed modules into one consistent simulation flow. The next steps are to connect memory state and dirty-page/write-back counts to the selected algorithms, expose thrashing/working-set analysis through the main program, run end-to-end tests on manual and file traces, compare all three policies, and document any remaining limitations. Additional testing should cover multiple processes and trace-format behavior if required by the project proposal.

7. Repository

GitHub: https://github.com/Layan19-2004/ICS433-Virtual-Memory-Simulator